

Projeto 2: Algoritmos para Grafos

Solução Ótima para o 8-Puzzle e 15-Puzzle utilizando Buscas Heurísticas (IDA* e A*)

Gabriel Davi; Giordani Andre

Professor: Tadeu Zubaran

[Link GitHub](#)

[Link Planilha](#)

9 de junho de 2026

1 Introdução e Definição do Problema

Este relatório descreve a implementação de uma solução computacional baseada em grafos para os clássicos problemas do 8-Puzzle e 15-Puzzle. O objetivo principal é encontrar o caminho ótimo (menor número de movimentos) a partir de um estado embaralhado até o estado objetivo ordenado.

O desafio inerente a este problema é a explosão combinatória do espaço de estados. Enquanto o 8-Puzzle possui um espaço tratável de $9!/2 = 181.440$ estados válidos, o 15-Puzzle expande-se para $16!/2 \approx 10,4 \times 10^{12}$ estados. A busca cega em grafos dessa magnitude é computacionalmente inviável, tornando mandatória a aplicação de algoritmos de Busca Informada (A* e IDA*) guiados por heurísticas admissíveis para realizar a poda da árvore de busca.

2 Fundamentação Teórica e Estado da Arte

A construção da solução inicial utilizou a métrica de **Distância de Manhattan combinada com Conflito Linear**. Esta abordagem atua calculando a distância física mínima de cada peça ao seu destino e penalizando adicionalmente peças na mesma linha ou coluna que precisem "saltar" umas sobre as outras (engarrafamentos).

Para superar o limite de tempo em instâncias complexas do 15-Puzzle, a literatura introduz o uso de **Pattern Databases (PDBs)**. Um PDB pré-computa e armazena em memória os caminhos ótimos exatos para subconjuntos de peças. O diferencial técnico desta implementação é o uso de *Disjoint Pattern Databases* associado a uma heurística híbrida, equilibrando de forma rigorosa o consumo de memória (RAM) e a velocidade de busca.

3 Desenvolvimento e Ferramentas de Otimização

O motor de resolução foi projetado para maximizar a performance através de técnicas avançadas de otimização:

- **IDA* (Iterative Deepening A*)**: Em vez do clássico A*, que armazena toda a fronteira de busca na memória (levando frequentemente a estouros de RAM no 15-Puzzle), implementamos o IDA*. Ele utiliza busca em profundidade com limites incrementais,

consumindo memória na ordem de $O(L)$ (onde L é o tamanho do caminho), garantindo resiliência em instâncias complexas.

- **Geração de PDB com Custo Amortizado:** O algoritmo indexa o custo das partições na primeira execução, gravando os dados em ficheiros binários (.dat). Nas execuções subsequentes, o tempo de carga é reduzido a operações de I/O instantâneas, tornando o acesso à heurística um custo de complexidade estrita $O(1)$.
- **Heurística Híbrida e Prevenção de *Double Counting*:** Para lidar com os "pontos cegos" do PDB no 15-Puzzle, o sistema consulta simultaneamente o Banco de Padrões e a função clássica de Conflito Linear. Crucialmente, os valores não são somados, mas submetidos a um `std::max(Total PDB, Total Clássico)`. Isso evita o *Double Counting* (penalizar a mesma configuração de peças bloqueadas duas vezes, pois o PDB já contém engarrafamentos internos), mantendo a garantia matemática de admissibilidade ao mesmo tempo que assegura a maior poda possível.
- **Filtro de Viabilidade (Paridade e Inversões):** Sabe-se que metade do espaço de estados do 15-Puzzle é matematicamente insolúvel. O sistema implementa uma validação em tempo constante $O(1)$ antes do início da busca, cruzando a soma das inversões das peças com a paridade da linha do espaço vazio. Isso impede que o algoritmo entre em *loops* exaustivos tentando resolver instâncias impossíveis.
- **Dimensionamento Exato de Memória:** A alocação da RAM foi calculada de forma preditiva por análise combinatória. No 15-Puzzle (16 posições), alocar um grupo de 6 peças resulta em $16 \times 15 \times 14 \times 13 \times 12 \times 11 = 5.765.760$ permutações. Armazenando o custo em inteiros de 1 byte (`uint8_t`), cada subconjunto ocupa estritamente $\approx 5,5$ MB. Em contraste, o 8-Puzzle mapeia todo o tabuleiro (9 posições para 8 peças), totalizando $9! = 362.880$ estados (≈ 354 KB), justificando a diferença arquitetural entre os PDBs de cada instância.

4 Análise de Resultados Computacionais

4.1 Admissibilidade e Oráculo no 8-Puzzle

O dado fundamental extraído da execução no 8-Puzzle é o comportamento do PDB como um verdadeiro "Oráculo" ($h = h^*$). A média de movimentos ótimos foi idêntica ao número de nós expandidos (22,16). O algoritmo não precisou realizar deduções exploratórias; ele navegou diretamente para a solução, comprovando matematicamente que a heurística mapeou a totalidade do espaço de estados sem incorrer em subestimativas.

4.2 O Poder da Poda (Pruning)

A Tabela 1 ilustra a drástica redução na exploração de grafos ao transitar da heurística puramente calculada em tempo real para a arquitetura baseada em memória.

Métrica Média	Manhattan + Conflito	PDB Híbrido	Redução de Nós
8-Puzzle (Nós Exp.)	1.123,74	22,16	-98,03%
15-Puzzle (Nós Exp.)	22.216.800	12.826.900	-42,26%

Tabela 1: Comparativo de nós expandidos evidenciando o poder de poda das heurísticas.

Ao poupar o algoritmo de expandir dezenas de milhões de nós desnecessários, a árvore de busca torna-se significativamente mais enxuta, evitando becos sem saída computacionais.

4.3 Custo Amortizado: A Troca de Espaço por Tempo

Um dos fenômenos mais vitais comprovados pela experimentação foi o *Space-Time Tradeoff* nas execuções do 8-Puzzle (Tabela 2).

Execução (8-Puzzle)	Nós Expandidos	Tempo Médio/Inst.	Melhoria de Tempo
Manhattan (Baseline)	1.123,74	0,12089 ms	-
PDB (1ª Execução - Criação)	22,16	0,14029 ms	+16,05% (Piora)
PDB (2ª Execução - Leitura)	22,16	0,00259 ms	-97,86% (Melhoria)

Tabela 2: Análise de Custo Amortizado no 8-Puzzle demonstrando o lucro computacional da pré-computação.

A primeira chamada do PDB apresenta um ligeiro atraso temporal devido à sobrecarga de pré-computação e escrita no disco (construção do grafo parcial). Contudo, este é um imposto pago apenas uma vez. Na segunda execução, o sistema acessa os dados pré-carregados, dizimando o tempo de busca em quase 98%.

Para atestar o comportamento desse fenômeno em cenários de alta complexidade, os mesmos testes de reexecução foram conduzidos no 15-Puzzle (Tabela 3).

Média (15-Puzzle)	Movimentos	Nós Expandidos	Tempo Total (s)
Manhattan (Baseline)	53,05	2,22E+07	3.808,72
PDB (1ª Execução)	53,05	1,28E+07	3.036,08
Melhoria (1ª Exec. vs Base)	0,00%	-42,26%	-20,29%
PDB (2ª Execução)	53,05	1,28E+07	2.914,94
Melhoria (2ª Exec. vs Base)	0,00%	-42,26%	-23,47%

Tabela 3: Desempenho no 15-Puzzle (IDA*): A diluição do tempo de criação do PDB frente ao tempo total de busca.

Uma observação fascinante e matematicamente sólida surge ao analisar a Tabela 3. Diferentemente do 8-Puzzle, onde a primeira execução sofre uma penalidade percentual notória, no 15-Puzzle a sobrecarga de criação das tabelas em memória torna-se irrisória face ao tempo massivo de exploração exigido pelo problema.

Mesmo pagando o custo de I/O para gerar as partições no disco (1ª Execução), o PDB ainda garante uma melhoria expressiva de 20,29% no tempo total. A diferença de aproximadamente 121 segundos entre a 1ª e a 2ª execução reflete o custo exato dessa pré-computação. Esse custo fixo de ~ 2 minutos é completamente diluído nos quase 50 minutos de processamento bruto. Na execução final, operando estritamente em memória (2ª Execução), o PDB consolida uma economia máxima de tempo no IDA*, superando a abordagem Manhattan clássica em 23,47%.

4.4 O Triunfo do A* e a Eficiência em RAM

A grande vitória da otimização via PDB consolidou-se ao reintroduzirmos o algoritmo A* clássico. Inicialmente descartado no 15-Puzzle devido ao seu consumo de memória exponencial $O(b^d)$ que causava falhas de alocação (`std::bad_alloc`), a redução de 42,26% no espaço de

estados providenciada pelo PDB tornou a árvore de busca pequena o suficiente para caber na memória principal.

Ao operar estritamente em RAM, o A* utilizou a sua *Closed List* para detetar transposições instantaneamente e evitou o maior gargalo do IDA*: o recálculo dos níveis iniciais a cada iteração do limite. O resultado empírico demonstrou que o A* com PDB resolveu as 100 instâncias em **2.576 segundos**, superando o melhor tempo do IDA* (2.914 segundos) em aproximadamente **11,6%**, estabelecendo a performance máxima do motor de resolução.

5 Conclusão

O projeto validou empiricamente os fundamentos da Inteligência Artificial em buscas informadas. O uso rigoroso da admissibilidade na combinação `std::max(PDB, Conflito Linear)` e a arquitetura IDA* asseguraram o mapeamento do caminho perfeitamente ótimo em 100% das 100 instâncias processadas.

Os testes confirmaram o postulado de que o dispêndio de memória volátil controlada reverte-se em economias massivas de ciclos de processamento. Ao mitigar a miopia heurística e podar mais de 42% da árvore de busca no 15-Puzzle, a solução provou ser uma mitigação direta e altamente eficiente para o problema da explosão combinatória inerente aos domínios NP-Difícil, culminando no regresso triunfal do A* como o algoritmo mais célere do projeto.